Before discovering HTMX (see chapter 12), my go-to stack was Vue.js and Django. This allowed me to use regular web pages, where it was easiest, and sprinkle in Vue.js for interactivity. For me, at least, this was a faster method of creating applications than a pure SPA. It still needs an API, though. When Vue.js interacts with a page, it sometimes needs information from the backend.

11.1.1 CRUD and REST

Back in chapter 5, you learned about *CRUD*—that's *create*, *read*, *update*, and *delete*. It is still a fun acronym, and it is applicable in more ways than just the context of the Django Admin. Any time you're dealing with data, those four operations drive the interaction. This applies to APIs as well. All APIs implement at least one of these operations, and many provide an interface for all four. You can think of an API as comprising "nouns" and "verbs," where the nouns are the objects getting acted upon and the verbs are one or more of the CRUD operations.

A noun/object in an API call may not be the same as the object in your backend—in fact, it is almost always different. Usually, a data transformation has to happen between the actual object and its representation in the API. This might seem messy at first, but it is no different from what the ORM does for you: it abstracts away a row in the database. As the consumer of your API may not be using Django, and in fact, they may not even be using Python, an intermediary data representation is necessary. The process of changing your actual object into this representation is known as *serialization*.

The rules that bind the interaction of two machines is called a *protocol*, and you're already using one. Your browser on one machine and the web server on another talk using HTTP. The GET and POST HTTP methods you learned about in chapter 7 are part of this protocol. A natural way of building an API is to use the existing HTTP protocol, rather than re-invent the wheel. This isn't required, as there are APIs out there that have their own protocols, but for a web-based scenario, HTTP makes the most sense.

By far, the most common API protocol in the web world is *representational state transfer* (REST). Calling REST a protocol is actually an overstatement; it is a loosely defined set of conventions that has become a de facto standard. It is built on top of HTTP, though, and its lack of strictness is probably one of the influencing factors of its wide adoption.

REST uses the HTTP methods to enact the verbs in CRUD. You've already seen GET for read and POST for create or update within the context of forms. There are other HTTP methods as well, though, and the three that typically get used with REST are PUT, PATCH, and DELETE. When implementing REST, POST typically gets reserved for creation only, with PUT and PATCH being for updates. The difference between the two is based on what information gets provided. PUT expects all an object's fields, while PATCH expects only those that have changed. HTTP DELETE, quite sensibly, gets used for deleting.